

## 상속과 다형성을 활용한 던제 전투

시연 포함 목표 8분 45초. 실제 시간은 말하는 속도에 따라 달라집니다. 이 대본은 예시이므로 이해한 내용에 맞게 표현을 조정하세요.

이번에는 게임 캐릭터 예제에 실제 체력이 줄어드는 던제 전투를 추가했습니다. 중심 학습 내용은 상속과 오버라이딩이고, 다형성을 추가로 응용했습니다. 모든 코드를 설명하기보다는 공격 메서드의 반환값이 전투에 어떻게 연결되는지 핵심 흐름을 소개하겠습니다.

## 오늘 발표할 내용

오늘 발표는 네 부분으로 진행하겠습니다. 먼저 어떤 프로그램인지와 부모·자식 클래스의 관계를 간단히 보겠습니다. 다음으로 가장 중요한 코드 세 부분을 설명하겠습니다. 공격 메서드가 값을 반환하는 부분, 서로 다른 직업을 같은 변수로 다루는 부분, 받은 피해량을 체력에 적용하는 부분입니다. 이후 프로그램을 짧게 실행해 코드가 실제로 어떻게 동작하는지 확인하겠습니다. 마지막으로 헛갈리는 부분과 코드에서 개선하고 싶은 점을 정리하겠습니다. 전체 발표는 시연을 포함해 약 9분입니다.

## 확장한 프로그램의 흐름

전사와 마법사에 궁수와 도적을 추가해 직업을 네 개로 늘렸습니다. 전사는 높은 체력, 마법사는 강한 특수 공격, 궁수는 범위 안에서 무작위 피해, 도적은 확률에 따른 치명타로 차이를 두었습니다. 직업을 고르면 능력치를 먼저 확인하고 확정하거나 다른 직업을 다시 고릅니다. 확정된 뒤 일반 공격, 특수 공격, 회복 중 하나를 선택합니다. 몬스터가 살아 있으면 반격하고 한쪽 체력이 0이 되면 끝납니다. 특수 공격과 물약은 각각 두 번 사용할 수 있습니다.

## 상속과 오버라이딩의 역할

핵심 코드를 보기 전에 클래스 관계를 먼저 정리하겠습니다. GameCharacter는 부모 클래스이며 이름과 체력, 공격력처럼 모든 직업에 필요한 값을 갖습니다. takeDamage와 heal처럼 공통으로 사용하는 메서드도 여기에 있습니다. Warrior, Mage, Archer, Rogue는 이를 상속하는 자식 클래스입니다. 화면의 extends GameCharacter가 그 관계를 나타냅니다. 상속 덕분에 각 직업에서 체력 감소와 회복 코드를 모두 다시 작성할 필요가 없습니다. 다만 공격 방식은 직업마다 다르기 때문에 attack과 specialAttack을 자식 클래스에서 다시 작성했습니다. 이것이 오버라이딩입니다. 상속은 공통 기능을 물려받는 관계이고, 오버라이딩은 물려받은 메서드의 동작을 바꾸는 방법으로 구분하면 됩니다. 다음 장에서는 마법사의 특수 공격을 예로 보겠습니다.

## 핵심 1. 공격 메서드의 반환값

대표로 마법사의 특수 공격 코드를 보겠습니다. 부모의 `specialAttack`을 같은 이름으로 재정의했습니다. 기존 예제는 `void` 메서드에서 출력만 했지만 이번에는 `int` 반환형을 사용합니다. 지역 변수 `damage`에 기본 공격력의 두 배를 저장하고, 마지막에 `return damage`로 호출한 곳에 전달합니다. 출력은 화면에 보여주는 일이고 `return`은 다른 코드가 사용할 값을 전달하는 일이라는 차이가 있습니다. 이 계산만으로 몬스터 체력이 바뀌는 것은 아니고, 체력 변경은 `Main`에서 처리합니다.

코드를 위에서 아래로 짚으며 설명합니다. 마법사의 `attackPower`가 22이므로 `damage`에는 44가 들어갑니다. 화면에 44를 출력하는 줄과 `return`으로 44를 보내는 줄을 각각 가리킵니다. 반환형이 `int`인 메서드는 호출한 코드에 정수 결과를 돌려줍니다. 이 메서드 안에는 `monsterHp`를 바꾸는 줄이 없다는 점도 확인합니다.

## 핵심 2. 직업이 달라도 같은 호출

다형성은 추가 학습으로 넣은 부분입니다. player는 부모 GameCharacter 타입으로 선언합니다. 직업 선택에서는 전사나 마법사 같은 실제 자식 객체를 연결합니다. 공격하는 곳에서는 직업별 조건문을 다시 쓰지 않고 player.attack을 호출합니다. 실제 객체가 궁수면 궁수의 공격, 도적이면 도적의 공격이 실행됩니다. 화면에는 두 직업만 발췌했지만 같은 방식으로 네 직업을 처리합니다. 상속으로 공통 타입을 만들고 오버라이딩으로 다른 공격을 작성한 결과를 여기서 활용합니다.

player 변수는 하나이고 사용자가 고른 직업의 객체 하나가 연결됩니다. 전사를 골랐을 때와 마법사를 골랐을 때를 각각 손으로 짚어 보겠습니다. 두 경우 모두 마지막 호출문은 player.attack()으로 같습니다. Main은 반환된 피해량만 받아 사용합니다. 직업에 따른 공격 계산은 각 자식 클래스가 맡습니다.

## 핵심 3. 피해량으로 전투 상태 변경

앞에서 반환받은 damage를 몬스터 체력에서 뺍니다. 이 부분이 공격 계산을 실제 전투 상태와 연결합니다. 그 다음 몬스터 체력이 0 이하인지 먼저 검사합니다. 처치했다면 break로 전투 반복을 끝내므로 반격 코드에 도달하지 않습니다. 살아 있을 때만 반격 피해를 정하고 player.takeDamage를 호출합니다. 공격, 처치 확인, 반격이라는 실행 순서가 중요합니다. 예를 들어 체력 120인 몬스터에게 화염구 44 피해를 주면 남은 체력은 76이 됩니다.

monsterHp가 30이고 피해량이 44라면 뺀 결과는 -14입니다. 조건문이 이를 0으로 바꾸고 break로 반복문을 끝냅니다. 그래서 아래 반격은 실행하지 않습니다. Math.random()은 0 이상 1 미만의 값을 만들고, 8을 곱해 정수로 바꾸면 0부터 7, 여기에 14를 더하면 14부터 21이 됩니다. 이 숫자는 몬스터의 반격 피해량입니다.

## 실행으로 확인할 부분

시연에서는 전사를 골라 특수 공격을 두 번 사용해 보겠습니다. 한 번에 36씩 피해를 주므로 몬스터 체력은 120에서 84, 다시 48이 됩니다. 세 번째 특수 공격은 횡수가 없다는 안내만 나오고 반격하지 않습니다. 회복도 물약이 있을 때만 가능하고 최대 체력을 넘지 않습니다. 마지막 공격으로 몬스터를 처치하면 승리 안내가 나옵니다. 반격 수치는 난수여서 실행마다 남은 체력은 달라질 수 있습니다.

### 시연 조작 (읽지 않는 메모)

직업 1 → 확정 1 → 특수 공격 2 → 특수 공격 2 → 다시 2(횡수 부족 확인) → 일반 공격 1 → 회복 3 → 일반 공격 1 → 일반 공격 1

직업 확정 전에는 전투가 시작되지 않습니다. 전사 특수 공격 후 몬스터 체력 84, 48을 확인합니다. 마지막 공격 뒤 반격 없이 승리하는지 보여 주세요. 화면 전환과 입력을 포함해 약 50초를 사용합니다.



## 어려운 부분과 복습할 순서

어려운 부분은 메서드 호출과 반환이 연결되는 흐름입니다. 자식 클래스 안의 damage와 Main의 damage는 이름은 같아도 서로 다른 지역 변수입니다. return으로 값을 전달해야 Main이 받아서 사용할 수 있습니다. 또 전투에서는 조건문의 순서를 놓치면 처치된 몬스터가 반격하는 문제가 생길 수 있습니다. 그래서 복습할 때는 공격 하나를 정하고 자식의 계산, 반환, Main의 체력 감소, 처치 확인 순서로 따라가려고 합니다.

복습할 때는 변수 이름을 외우는 것보다 값이 어디에서 계산되고 어디에서 사용되는지 표시해 보려고 합니다. 예를 들어 메서드 내부 damage가 44이고 return으로 보내면 Main의 damage가 그 값을 받는 과정을 종이에 적어 볼 수 있습니다.

## 코드의 아쉬운 점과 개선 방향

기능을 추가하면서 Main에 입력 처리와 전투 처리가 함께 모여 길어졌습니다. 다음에는 한 턴 처리나 직업 선택을 메서드로 나누면 읽기 쉬울 것 같습니다. 또 능력치 확인 화면에는 피해량을 설명 문구로 적었고, 실제 공격은 자식 클래스에서 따로 계산합니다. 예를 들어 전사의 공격 보너스를 바꾸면 안내 문구도 같이 고쳐야 해서 빠뜨릴 수 있습니다. 직업별 설명 메서드로 옮기고 같은 수치를 함께 사용하면 좋을 것 같습니다. 이번 발표에서는 모든 문법보다 같은 공격 호출과 반환값이 실제 전투를 움직이는 흐름에 집중했습니다.

먼저 현재 동작을 이해한 뒤 한 부분씩 나누고, 수정 전후의 실행 결과를 비교하는 순서로 개선하려고 합니다.

## 발표 전 복습할 포인트

### 상속과 생성자

Mage extends GameCharacter는 부모의 기능을 물려받는 관계입니다. `super("마법사", 85, 22)`는 부모 생성자를 호출해 공통 필드를 초기화합니다.

### 오버라이딩과 다형성

자식에서 `attack()`을 다시 작성하는 것이 오버라이딩입니다. GameCharacter player에 자식 객체를 연결하고 `player.attack()`으로 실제 자식 메서드를 실행하는 활용이 다형성입니다. 다형성은 추가 학습으로 구분합니다.

### 출력과 반환

`println`은 화면에 보여 주고, `return`은 호출한 곳으로 값을 돌려줍니다. Mage 안의 `damage`와 Main 안의 `damage`는 별개의 지역 변수입니다.

### 반복문 제어

`break`는 가장 가까운 반복문 하나만 끝냅니다. `confirmed`가 `true`일 때 바깥 직업 선택 반복도 끝납니다. `continue`는 현재 반복의 나머지를 건너뛰므로 잘못된 입력에는 반격하지 않습니다.

### 직접 설명해 볼 질문

왜 능력치 확인 화면에서 `attack()`을 실행하지 않을까? 몬스터 처치 확인을 반격 뒤로 옮기면 어떻게 될까? 전사 공격 보너스를 바꾸면 어느 설명도 고쳐야 할까?

## 코드 검토와 다음 개선 과제

### 현재 구현 범위

일반 클래스 6개, 생성자, 상속, 오버라이딩, 다형성, Scanner, if, while, 기본 연산과 Math.random을 사용합니다. 프레임워크, 외부 라이브러리, 컬렉션, 스트림, 예외 처리 구조는 추가하지 않았습니다.

### 1. Main이 길어짐

직업 선택, 능력치 안내, 확정 메뉴, 전투가 main에 모여 있습니다. 흐름을 위에서 아래로 따라가기는 쉽지만 수정할 위치를 찾기 어렵습니다. 다음에는 직업 선택과 한 턴 처리를 메서드로 분리할 수 있습니다.

### 2. 안내와 계산의 중복

Main의 피해량 설명과 자식의 공격 계산식을 따로 관리합니다. 현재 값은 일치하지만 보너스 수치를 변경하면 안내도 함께 고쳐야 합니다. 직업별 설명 메서드와 공통 수치로 관리하는 것이 다음 개선 방향입니다.

### 3. 필드 접근과 숫자

hp와 attackPower는 같은 패키지에서 직접 접근할 수 있습니다. private과 필요한 메서드로 관리할 수 있습니다. 회복량 30 등은 뜻이 드러나는 이름으로 정리할 수 있습니다.

### 공부 내용을 말할 때

실제로 연습한 뒤 “같은 호출이 직업에 따라 다르게 실행되는 것을 확인했습니다”처럼 구체적으로 말하세요. 아직 겪지 않은 오류를 해결했다고 말하기보다, 현재 헛갈리는 반환 흐름과 개선 방향을 설명하면 됩니다.